

Databricks 14 Days AI Challenge

Day 4: Delta Lake Transformation

1. What is Delta Lake?

Delta Lake is an open-source storage layer that adds reliability, versioning, and ACID guarantees on top of data lakes. It tracks file updation(addition, deletion) , version history and schema changes.

Aspect	Normal Data Lake	Delta Lake
Storage type	Just files in cloud storage	Parquet files + transaction log
File format	CSV, JSON, Parquet, etc.	Parquet (optimized with metadata layer)
Transaction guarantees	No ACID guarantees	Full ACID support
Data corruption risk	Easy to corrupt with concurrent writes	Safe concurrent reads and writes
Schema control	No strict enforcement	Schema enforcement and validation
Schema evolution	Manual handling	Built-in schema evolution support
Version control	No version history	Built-in versioning
Time travel	Not supported	Supported (query previous versions)
Reliability	Basic storage	Production-grade reliability
Best for	Raw storage	Reliable data pipelines and analytics

2. ACID transactions

ACID = Atomicity, Consistency, Isolation, Durability

Scenario: Two jobs write to same table at same time.

Result: Corrupted files, Partial writes, Duplicate data and Broken pipelines

Because Parquet alone has no transaction management.

✓ With Delta

Delta uses its transaction log to:

- Lock writes
- Track versions
- Ensure atomic commits
- Prevent data corruption

Example: `df.write.format("delta").mode("append").save("/mnt/delta/sales")`

If write fails midway: Delta rolls back and Table remains consistent

That's atomicity.

Real-world example

Imagine:

- 10 Spark jobs running
- All writing to same table
- Streaming + batch mixed

Delta ensures:

- No half-written data
- No dirty reads
- No race conditions

This is why it's used in production.

3. Schema Enforcement

Parquet behavior:

If you write inconsistent schema:

df1 columns: id, name

df2 columns: id, age

Parquet: Will allow it, Merge weirdly and Break downstream queries

No strict enforcement.

Delta behavior

If schema doesn't match:

It throws an error.

Example: `df.write.format("delta").mode("append").save("/mnt/delta/users")`

If schema changes, you'll see: `AnalysisException: A schema mismatch detected`

That's **protection**.

Optional: Schema Evolution

If you *intentionally* want schema changes:

```
df.write.format("delta") \
    .option("mergeSchema", "true") \
    .mode("append") \
    .save("/mnt/delta/users")
```

Delta tracks it safely in `_delta_log`.

4. Delta vs Parquet

Feature	Parquet	Delta Lake
Storage format	Columnar	Columnar (Parquet underneath)
ACID transactions	✗ No	✓ Yes
Schema enforcement	✗ Weak	✓ Strong
Time travel	✗ No	✓ Yes
Concurrent writes	✗ Risky	✓ Safe
Streaming support	Limited	Native support
Updates / Deletes	Hard	Supported

Parquet:

You cannot do: DELETE FROM table WHERE id = 10

Delta:

You can: DELETE FROM sales WHERE id = 10;

Because Delta rewrites files and logs changes.

5. Time Travel (Delta Superpower)

You can query previous versions:

```
spark.read.format("delta") \  
  .option("versionAsOf", 2) \  
  .load("/mnt/delta/sales")
```

This is insanely powerful for:

- debugging pipelines
- auditing
- rollback

Parquet cannot do this.

Real-world scenario: Imagine: Daily sales table and ETL bug corrupts data

With Parquet: You panic and restore from backup (if any)

With Delta: Problem solved

RESTORE TABLE sales TO VERSION AS OF 5;